

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#6)

Use of Venus Simulator for RISC-V Procedure Execution

Student Name: Abdulmoeiz
Roll No: 241-6510
Section: CE_A

Date performed: 29,09, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

If a procedure needs an s register, it must **save it on the stack at entry and restore it before returning**. The procedure overwrites s0, which is callee-saved, without restoring it. main expected s0 to still be 2500. main has no exit call. After the print ecall, execution falls into procedure, runs it again, and jalr returns to ra, which loops forever. We need an exit ecall

Code:

```
.text
main:
    li a0, 1
    li s0, 2500
    jal ra, procedure
    add t1, s0, a0    # t1 = 2500 + 1003 = 3503
    li a0, 1
    mv a1, t1
    ecall
    li a0, 10
    ecall
procedure:
    addi sp, sp, -4
    sw s0, 0(sp)    # save caller's s0
    li s0, 3
    li t0, 1000
    add a0, s0, t0    # a0 = 1003
    lw s0, 0(sp)    # restore s0 = 2500
    addi sp, sp, 4
    jalr zero, ra, 0
```

Output:

PC	Machine Code	Basic Code	Original Code	(x3)
0x0	0x00100513	addi x10 x0 1	li a0, 1	tp 0
0x4	0x00001437	lui x8 1	li s0, 2500	(x4) t0 1000
0x8	0x9C440413	addi x8 x8 -1596	li s0, 2500	(x5) t1 3503
0xc	0x01C000EF	jal x1 28	jal ra, procedure	(x6) t2 0
0x10	0x00A40333	add x6 x8 x10	add t1, s0, a0 # t1 = 2500 + 1003 = 3503	(x7) s0 2500
0x14	0x00100513	addi x10 x0 1	li a0, 1	(x8) s1 0
...	...	...	...	(x9)

Task 2:

Description:

This task runs a short program in Venus that shows how the stack works. Three register values are stored on the stack using sw, then loaded back into different registers using lw. The stack pointer is moved down to make room and moved back up at the end.

Code:

```
.text
main:
    addi s0, zero, 5    # s0 = 0 + 5 = 5
    addi t1, zero, 10   # t1 = 0 + 10 = 10
    addi t0, zero, 15   # t0 = 0 + 15 = 15
    addi sp, sp, -12    # sp = sp - 12
    sw    s0, 0(sp)     # Mem[sp + 0] = s0 = 5
    sw    t0, 4(sp)     # Mem[sp + 4] = t0 = 15
    sw    t1, 8(sp)     # Mem[sp + 8] = t1 = 10
    lw    t4, 8(sp)     # t4 = Mem[sp + 8] = 10
    lw    t2, 4(sp)     # t2 = Mem[sp + 4] = 15
    lw    t3, 0(sp)     # t3 = Mem[sp + 0] = 5
    addi sp, sp, 12     # sp = sp + 12
    li    a0, 10        # a0 = 10
    ecall               # terminate
```

Output:

PC	Machine Code	Basic Code	Original Code	(x3)
0x0	0x00500413	addi x8 x0 5	addi s0, zero, 5 # s0 = 0 + 5 = 5	tp 0
0x4	0x00A00313	addi x6 x0 10	addi t1, zero, 10 # t1 = 0 + 10 = 10	(x4) t0 15
0x8	0x00F00293	addi x5 x0 15	addi t0, zero, 15 # t0 = 0 + 15 = 15	(x5) t1 10
0xc	0xFF410113	addi x2 x2 -12	addi sp, sp, -12 # sp = sp - 12	(x6) t2 15
0x10	0x00812023	sw x8 0(x2)	sw s0, 0(sp) # Mem[sp + 0] = s0 = 5	(x7) s0 5
				(x8) s1 0
				(x9)

Task 3:

Description:

Here we write a procedure that takes two numbers, a and b, and returns $(a+b) \times (a-b)$. The arguments are passed in a0 and a1, and the result comes back in a0. main calls the procedure and prints the answer.

Code:

```
.text

main:
    li a0, 7      # a = 7
    li a1, 3      # b = 3
    jal ra, EqSolver
    addi s0, a0, 0    # s0 = a0 + 0
    addi a1, s0, 0    # a1 = s0
    li a0, 1
    ecall
    li a0, 10
    ecall
EqSolver:
    add t0, a0, a1    # t0 = a + b
    sub t1, a0, a1    # t1 = a - b
    mul a0, t0, t1     # a0 = (a+b) * (a-b)
    jalr zero, ra, 0
```

Output:

PC	Machine Code	Basic Code	Original Code	gp (x3)
0x0	0x00700513	addi x10 x0 7	li a0, 7 # a = 7	268435456
0x4	0x00300593	addi x11 x0 3	li a1, 3 # b = 3	tp (x4)
0x8	0x01C000EF	jal x1 28	jal ra, EqSolver	t0 10
0xc	0x00050413	addi x8 x10 0	addi s0, a0, 0 # s0 = a0 + 0	(x5)
0x10	0x00040593	addi x11 x8 0	addi a1, s0, 0 # a1 = s0	t1 4
0x14	0x00100513	addi x10 x0 1	li a0, 1	(x6)
0x18	0x00000073	ecall	ecall	t2 0
				(x7)
				s0 40
				(x8)
				s1 0

Task 4:

Description:
In this task, we write two procedures, AreaFinder and PerimeterFinder, that take the side of a square. One returns the area and the other returns the perimeter .main calls both and prints the results
Code:
<pre>.text main: li s0, 5 # s0 = side = 5 addi a0, s0, 0 # a0 = side jal ra, AreaFinder addi s1, a0, 0 # s1 = area addi a0, s0, 0 jal ra, PerimeterFinder # returns perimeter in a0 addi s2, a0, 0 # s2 = perimeter addi a1, s1, 0 # a1 = area li a0, 1 ecall li a1, 10 li a0, 11 ecall addi a1, s2, 0 # a1 = perimeter li a0, 1 # print integer ecall li a0, 10</pre>

ecall

AreaFinder:

```
mul a0, a0, a0    # area = side * side
jalr zero, ra, 0  # return
```

PerimeterFinder:

```
slli a0, a0, 2    # perimeter = 4 * side
jalr zero, ra, 0
```

Output:

PC	Machine Code	Basic Code	Original Code	gp (x3)	268435456
0x0	0x00500413	addi x8 x0 5	li s0, 5 # s0 = side = 5	tp (x4)	0
0x4	0x00040513	addi x10 x8 0	addi a0, s0, 0 # a0 = side	t0 (x5)	0
0x8	0x040000EF	jal x1 64	jal ra, AreaFinder	t1 (x6)	0
0xc	0x00050493	addi x9 x10 0	addi s1, a0, 0 # s1 = area	t2 (x7)	0
0x10	0x00040513	addi x10 x8 0	addi a0, s0, 0	s0 (x8)	5
0x14	0x03C000EF	jal x1 60	jal ra, PerimeterFinder # returns perimeter in a0	s1	25

Task 5:

Description:

This task asks for a procedure that calculates $x^3 + 5x^2 + 6x$ for a given x . To keep the code short, the expression is rearranged so that only two multiplications are needed. Main calls it and prints the result.

Code:

```
.text
main:
    li a0, 2    # x = 2
    jal ra, EqSolver
```

```

addi s0, a0, 0    # s0 = y
addi a1, s0, 0    # a1 = y
li a0, 1
ecall
li a0, 10
ecall
EqSolver:
addi t0, a0, 5    # t0 = x + 5
mul t0, t0, a0    # t0 = (x + 5) * x
addi t0, t0, 6    # t0 = (x + 5) * x + 6
mul a0, t0, a0    # a0 = ((x + 5) * x + 6) * x
jalr zero, ra, 0

```

Output:

Run
Step
Prev
Reset
Dump
Trace
Re-assemble from Editor

PC	Machine Code	Basic Code	Original Code
0x0	0x00200513	addi x10 x0 2	li a0, 2 # x = 2
0x4	0x01C000EF	jal x1 28	jal ra, EqSolver
0x8	0x00050413	addi x8 x10 0	addi s0, a0, 0 # s0 = y
0xc	0x00040593	addi x11 x8 0	addi a1, s0, 0 # a1 = y
0x10	0x00100513	addi x10 x0 1	li a0, 1
0x14	0x00000073	ecall	ecall
0x18	0x00A00513	addi x10 x0 10	li a0, 10

(x2)
gp 268435456
(x3)
tp 0
(x4)
t0 20
(x5)
t1 0
(x6)
t2 0
(x7)
s0 40
(x8)
s1 0
(x9)

Task 6:

Description:

In this task, main takes two values, n and k, and decides which function to call. If k is 1 it calls F1, which adds the squares of n and k. Otherwise, if n is greater than k, it calls F2, which returns the cube of n. If neither is true, it calls F3, which adds 5 to n. This practices if, else if and else using branch instructions.

Code:

```
.text
main:
    li s0, 6          # n = 6
    li s1, 2          # k = 2
    li t0, 1
    bne s1, t0, check_n_k # if (k != 1)
    addi a0, s0, 0      # a0 = n
    addi a1, s1, 0      # a1 = k
    jal ra, F1
    jal zero, done
check_n_k:
    bge s1, s0, else_case # if (k >= n)
    addi a0, s0, 0      # a0 = n
    jal ra, F2
    jal zero, done
else_case:
    addi a0, s0, 0      # a0 = n
    jal ra, F3
done:
    addi s2, a0, 0      # s2 = y
    addi a1, s2, 0      # print y
    li a0, 1
    ecall
    li a0, 10
    ecall
F1:
    mul t0, a0, a0      # t0 = x * x
    mul t1, a1, a1      # t1 = z * z
    add a0, t0, t1      # a0 = x^2 + z^2
    jalr zero, ra, 0
F2:
    mul t0, a0, a0      # t0 = z^2
    mul a0, t0, a0      # a0 = z^2 * z = z^3
    jalr zero, ra, 0
F3:
    addi a0, a0, 5      # a0 = w + 5
    jalr zero, ra, 0
```

Output:

Run
Step
Prev
Reset
Dump
Trace
Re-assemble from Editor

PC	Machine Code	Basic Code	Original Code
0x0	0x00600413	addi x8 x0 6	li s0, 6 # n = 6
0x4	0x00200493	addi x9 x0 2	li s1, 2 # k = 2
0x8	0x00100293	addi x5 x0 1	li t0, 1
0xc	0x00549A63	bne x9 x5 20	bne s1, t0, check_n_k # if (k != 1) go test n > k
0x10	0x00040513	addi x10 x8 0	addi a0, s0, 0 # a0 = n
0x14	0x00048593	addi x11 x9 0	addi a1, s1, 0 # a1 = k
...	...	...	...

gp (x3) 268435456
tp (x4) 0
t0 (x5) 36
t1 (x6) 0
t2 (x7) 0
s0 (x8) 6
s1 (x9) 2

Copy!
Download!
Clear!

Task 7:

Description:

This task is similar to the previous one, with different conditions. If k is 0 it calls fun1, which returns the square of n. Otherwise, if n equals k, it calls fun2, which returns the cube of n. If neither is true, it calls fun3, which adds 1 to n.

Code:

```

.text
main:
    li s0, 5      # n = 5
    li s1, 2      # k = 2
    bne s1, zero, check_eq # if (k != 0)
    addi a0, s0, 0 # a0 = n
    jal ra, fun1
    jal zero, done

check_eq:
    bne s0, s1, else_case # if (n != k)
    addi a0, s0, 0 # a0 = n
    jal ra, fun2
    jal zero, done
else_case:
    addi a0, s0, 0 # a0 = n

```

```

jal ra, fun3
done:
    addi s2, a0, 0      # s2 = y
    addi a1, s2, 0
    li a0, 1
    ecall
    li a0, 10
    ecall
fun1:
    mul a0, a0, a0
    jalr zero, ra, 0
fun2:
    mul t0, a0, a0      # t0 = z^2
    mul a0, t0, a0      # a0 = z^3
    jalr zero, ra, 0

fun3:
    addi a0, a0, 1
    jalr zero, ra, 0

```

Output:

PC	Machine Code	Basic Code	Original Code	
0x0	0x00500413	addi x8 x0 5	li s0, 5 # n = 5	(x3)
0x4	0x00200493	addi x9 x0 2	li s1, 2 # k = 2	tp (x4)
0x8	0x00049863	bne x9 x0 16	bne s1, zero, check_eq # if (k != 0)	t0 (x5)
0xc	0x00040513	addi x10 x8 0	addi a0, s0, 0 # a0 = n	t1 (x6)
0x10	0x038000EF	jal x1 56	jal ra, fun1	t2 (x7)
0x14	0x01C0006F	jal x0 28	jal zero, done	s0 (x8)
				s1

Task 8:

Description:

In this task, we find the nth Fibonacci number in two ways. The first uses a loop that keeps track of the last two numbers. The second is recursive, where fib of n calls fib of n minus 1 and fib of n minus 2, and uses

the stack to save the return address, n and the partial result. Both give the same answer, but the recursive one is slower and uses more stack memory

Code:

WITHOUT RECURSION)

```
.text
main:
    li a0, 5          # n = 5
    jal ra, fib
    addi s0, a0, 0
    addi a1, s0, 0
    li a0, 1
    ecall
    li a0, 10
    ecall

fib:
    bne a0, zero, init    # if (n != 0)
    jalr zero, ra, 0      # n == 0

init:
    li t0, 0             # a = 0
    li t1, 1             # b = 1
    li t2, 2             # i = 2

loop:
    blt a0, t2, end       # if (n < i)
    add t3, t0, t1         # c = a + b
    addi t0, t1, 0         # a = b
    addi t1, t3, 0         # b = c
    addi t2, t2, 1         # i++
    jal zero, loop

end:
    addi a0, t1, 0
    jalr zero, ra, 0
```

WITH RECURSION)

```
.text
main:
    li a0, 5          # n = 5
    jal ra, fib
    addi s2, a0, 0
    addi a1, s2, 0
    li a0, 1
```

```
ecall  
li a0, 10  
ecall
```

fib:

```
addi sp, sp, -12  
sw ra, 0(sp)  
sw s0, 4(sp)  
sw s1, 8(sp)
```

```
addi s0, a0, 0
```

```
beq s0, zero, proc # n == 0  
li t0, 1  
beq s0, t0, proc # n == 1  
addi a0, s0, -1 # a0 = n - 1  
jal ra, fib # a0 = fib(n-1)  
addi s1, a0, 0 # s1 = fib(n-1)
```

```
addi a0, s0, -2 # a0 = n - 2  
jal ra, fib # a0 = fib(n-2)
```

```
add a0, s1, a0 # a0 = fib(n-1) + fib(n-2)
```

proc:

```
lw ra, 0(sp)  
lw s0, 4(sp)  
lw s1, 8(sp)  
addi sp, sp, 12  
jalr zero, ra, 0
```

Output:

WITHOUT RECURSION)

Run

Step

Prev

Reset

Dump

Trace

Re-assemble from Editor

PC	Machine Code	Basic Code	Original Code
0x0	0x00500513	addi x10 x0 5	li a0, 5 # n = 5
0x4	0x01C000EF	jal x1 28	jal ra, fib
0x8	0x00050413	addi x8 x10 0	addi s0, a0, 0
0xc	0x00040593	addi x11 x8 0	addi a1, s0, 0
0x10	0x00100513	addi x10 x0 1	li a0, 1
0x14	0x00000073	ecall	ecall
0x18	0x00A00513	addi x10 x0 10	li a0, 10

t0 (x5)

3

t1 (x6)

5

t2 (x7)

6

s0 (x8)

5

s1 (x9)

0

a0 (x10)

10

a1 (x11)

5

WITH RECURSION)

Run

Step

Prev

Reset

Dump

Trace

Re-assemble from Editor

PC	Machine Code	Basic Code	Original Code
0x0	0x00500513	addi x10 x0 5	li a0, 5 # n = 5
0x4	0x01C000EF	jal x1 28	jal ra, fib
0x8	0x00050913	addi x18 x10 0	addi s2, a0, 0
0xc	0x00090593	addi x11 x18 0	addi a1, s2, 0
0x10	0x00100513	addi x10 x0 1	li a0, 1
0x14	0x00000073	ecall	ecall
0x18	0x00A00513	addi x10 x0 10	li a0, 10

(x7)

s0 (x8)

0

s1 (x9)

0

a0 (x10)

10

a1 (x11)

5

a2 (x12)

0

a3 (x13)

0

a4

0

Computer Organization & Architecture Lab

NUCES, ISLAMABAD

Page 15 of 15